CS213 – Assignment 2, Part 1

FINAL DESIGN & AI DESIGN PROCESS

Vole Machine Simulator | Mohammed Ramadan | Cairo University

Important note about conversation records

This report organizes the available conversation and saved Design 3 notes. Design 1 and Design 2 diagrams were pasted by the student in this chat. The saved Design 3 conversation document states it is an organized draft, not a verbatim export. Summaries are labeled as summaries and are not presented as exact transcripts. Attach actual chat screenshots/export if a complete original conversation is required.

1. Goal and requirements

Design an OOP UML model for a Vole Machine simulator: 256 memory cells, 16 registers, PC and IR, program loading, editable state, state display, step/run execution, validation, and screen output. No C++ implementation is included.

Design 1 — VoleMachine_v3 (student-provided)

Student input

The student shared a PlantUML design with ConsoleUI, ProgramLoader, VoleMachine, CPU, Instruction, ALU, Memory, RegisterFile, and Screen. VoleMachine owns Memory, RegisterFile, Screen, and CPU. CPU uses the machine state in step() and contains the current Instruction as its IR.

Assessment

Strengths: clear separation of user interface, loader, CPU, memory, registers, screen, and ALU; explicit runToHalt/step/reset methods; fixed-size memory and register storage; error cases are mentioned. Improvements: keep the CPU's access to collaborators consistent; make program validation and result reporting explicit; avoid relying on ambiguous screen-address examples without checking the official specification.

Design 2 — VoleSimulator (student-provided)

Student input

The student shared a second PlantUML design with VoleSimulator, CPU, Instruction, ProgramLoader, Memory, and RegisterFile. The simulator owns the core components; CPU stores PC, IR and halt status and references Memory and RegisterFile.

Assessment

Strengths: straightforward structure and separate methods for instruction families. Improvements: no separate Screen class, arithmetic/logic are grouped inside CPU, and the UI/editing/error-reporting responsibilities are not fully modeled. The raw pointer/reference ownership relationship should also be stated clearly.

Design 3 — ChatGPT proposal (from saved draft)

Prompt summary

Design an object-oriented class model for the Vole Machine simulator: 256 8-bit memory cells, 16 8-bit registers, PC and 16-bit IR, two-cell instructions, specified opcode families, file loading, editing memory/registers, state display, full or step-by-step execution, comment/blank-line handling, validation, screen output at FF, UML and PlantUML only; no implementation.

Response summary

The saved Design 3 draft proposes VoleMachine, CPU, Memory, RegisterFile, ProgramCounter, InstructionRegister, InstructionDecoder, DecodedInstruction, InstructionExecutor, ProgramLoader, ScreenDevice, ConsoleUI, StepResult,

ValidationResult, MachineSnapshot, and CPUSnapshot. It separates execution flow, decoding, execution, loading, output, and UI.

Follow-up question and answer — separation of responsibilities

Question recorded in the saved draft: Why separate CPU, InstructionDecoder, and InstructionExecutor? Answer summary: each class has a focused responsibility; VoleMachine coordinates, CPU controls execution flow, decoder interprets instruction fields, and executor performs the operation. This improves testability and reduces coupling.

Follow-up question and answer — execution flow

Question recorded in the saved draft: Explain the execution flow and HALT. Answer summary: ProgramLoader reads and validates; VoleMachine initializes state; CPU fetches two bytes into the IR; the instruction is decoded and executed; PC advances or jumps; results/state can be displayed; HALT marks execution stopped.

Follow-up question and answer — screen output

Question recorded in the saved draft: How should FF be handled? Answer summary: treat FF as memory-mapped output; a store targeting FF sends the value to ScreenDevice. The trigger and display format must be checked against the official assignment.

Follow-up question and answer — edge cases

Question recorded in the saved draft: Which edge cases should be included? Answer summary: file not found, malformed program text, program too large, invalid instruction/register operand, invalid memory access/jump target, clean HALT behavior, store to FF, stepping after halt, and reset/load consistency.

Final Design — refinement prompt and result

Prompt used for the final synthesis

Using the three candidate designs described above, produce a final UML class design for the Vole Machine simulator. Keep responsibilities clear and avoid unnecessary classes. Include the user interface, program loader, machine coordinator, CPU, instruction representation, memory, 16-register file, ALU, screen output, execution result and machine snapshot. Support loading at a valid even start address below FF, editing memory/registers, displaying memory/IR/registers/screen, single-step and run-to-halt, instruction validation, and HALT. Provide PlantUML only, not C++ implementation. Explicitly flag the assignment's possible FF versus 00 screen-output inconsistency instead of silently resolving it.

Final response summary

The final design keeps the clearer modularity of Design 1, the simple execution flow of Design 2, and the stronger separation of concerns proposed in Design 3. It retains a dedicated ALU and ScreenDevice, adds explicit StepResult and MachineSnapshot objects, and keeps PC, IR, and halted state inside CPU. ProgramLoader parses/validates file content; VoleMachine coordinates the components; ConsoleUI handles interaction only.

Final class responsibilities

ConsoleUI

Shows menu and machine state, asks the user to load/run/step/reset, edit a register or memory cell, and display memory, registers and screen output. It does not execute opcodes itself.

VoleMachine

Owns the main components and exposes reset, loadProgram, step, runToHalt, edit operations, halt status, and snapshot methods.

CPU

Stores PC, IR and halt state; fetches a 16-bit instruction from two adjacent cells; validates/decodes and executes it; updates PC or applies a jump; stops at HALT.

Instruction

Represents a 16-bit instruction and exposes opcode and operand fields.

Memory

Stores 256 8-bit cells and provides controlled read/write/clear/load/snapshot operations.

RegisterFile

Stores 16 8-bit registers and provides checked get/set/clear/snapshot operations.

ALU

Performs integer addition, floating-point addition, OR, AND, XOR and circular right rotation.

ScreenDevice

Collects output when a store targets the configured screen-mapped address.

ProgramLoader

Reads program text, ignores blank/comment lines, parses hex instructions, validates the start address and reports invalid input.

StepResult

Reports whether a step succeeded, whether the machine halted, and an explanatory message.

MachineSnapshot

Provides a read-only view of PC, IR, registers, memory, screen output and halt status for display.

Design rationale and limitations

Why this design

It is modular without creating separate classes for every tiny CPU field. The machine coordinator does not perform instruction decoding, the UI does not execute instructions, and ALU operations are independent. This improves readability, testing and later implementation consistency.

Validation required

Before final submission, compare opcode semantics and memory rules with the course specification and instructor/TA. The assignment's opcode table says address FF is the special screen-mapped address, while its sample program appears to store to address 00 for screen output. This document flags the inconsistency rather than assuming one behavior.

Academic integrity note

The assignment asks for actual AI prompts and conversations. The design descriptions and saved Design 3 file are organized notes/summaries, not a verified verbatim export. Attach actual chat screenshots or an exported conversation wherever exact history is required.

Final PlantUML source

Editable source is supplied separately as Final_Design.puml.

```
@startuml
title Vole Machine Simulator - Final Design
skinparam classAttributeIconSize 0
skinparam shadowing false
hide empty members

class ConsoleUI {
  - machine : VoleMachine&
  + run() : void
  - handleLoad() : void
  - handleRunAll() : void
  - handleStep() : void
  - handleReset() : void
  - handleRegisterEdit() : void
  - handleMemoryEdit() : void
  - displayState() : void
  - printMemoryMatrix() : void
  - printRegisters() : void
  - printScreen() : void
}

class VoleMachine {
  - memory : Memory
  - registers : RegisterFile
  - screen : ScreenDevice
  - cpu : CPU
  - loader : ProgramLoader
  + reset() : void
  + loadProgram(path : string, startAddress : uint8_t) : bool
  + step() : StepResult
  + runToHalt(maxSteps : int) : bool
  + isHalted() : bool
  + editRegister(index : uint8_t, value : uint8_t) : void
  + editMemory(address : uint8_t, value : uint8_t) : void
  + getSnapshot() : MachineSnapshot
}

class CPU {
  - pc : uint8_t
  - ir : Instruction
  - halted : bool
  + reset(startAddress : uint8_t = 0) : void
  + step(memory : Memory&, registers : RegisterFile&, screen : ScreenDevice&) : StepResult
  + getPC() : uint8_t
  + setPC(value : uint8_t) : void
  + getIR() : Instruction
  + isHalted() : bool
  - fetch(memory : Memory&) : void
  - decodeAndExecute(memory : Memory&, registers : RegisterFile&, screen : ScreenDevice&) : StepResult
}

class Instruction {
  - word : uint16_t
  + Instruction(word : uint16_t = 0)
  + raw() : uint16_t
  + opcode() : uint8_t
  + r() : uint8_t
  + x() : uint8_t
  + y() : uint8_t
  + xy() : uint8_t
  + isValid() : bool
  + toString() : string
}

class Memory {
  - cells : array<uint8_t, 256>
  + read(address : uint8_t) : uint8_t
  + write(address : uint8_t, value : uint8_t) : void
  + clear() : void
  + loadBytes(startAddress : uint8_t, bytes : vector<uint8_t>) : bool
  + snapshot() : vector<uint8_t>
}

class RegisterFile {
  - regs : array<uint8_t, 16>
  + get(index : uint8_t) : uint8_t
  + set(index : uint8_t, value : uint8_t) : void
  + clear() : void
  + snapshot() : vector<uint8_t>
}

class ScreenDevice {
  - outputBuffer : string
  + write(value : uint8_t) : void
  + getText() : string
  + clear() : void
}

class ProgramLoader <<utility>> {
  + {static} parseFile(path : string) : vector<uint8_t>
  + {static} validateStart(startAddress : uint8_t) : bool
}

class ALU <<utility>> {
  + {static} addInt(a : uint8_t, b : uint8_t) : uint8_t
  + {static} addFloat(a : uint8_t, b : uint8_t) : uint8_t
  + {static} bitOr(a : uint8_t, b : uint8_t) : uint8_t
  + {static} bitAnd(a : uint8_t, b : uint8_t) : uint8_t
  + {static} bitXor(a : uint8_t, b : uint8_t) : uint8_t
  + {static} rotateRight(value : uint8_t, steps : uint8_t) : uint8_t
}

class StepResult {
  + success : bool
  + halted : bool
  + message : string
}

class MachineSnapshot {
  + pc : uint8_t
```

```
  + ir : uint16_t
  + registers : vector<uint8_t>
  + memory : vector<uint8_t>
  + screenOutput : string
  + halted : bool
}

ConsoleUI --> VoleMachine : controls / displays
VoleMachine *-- "1" Memory
VoleMachine *-- "1" RegisterFile
VoleMachine *-- "1" ScreenDevice
VoleMachine *-- "1" CPU
VoleMachine ..> ProgramLoader : loads program
VoleMachine ..> MachineSnapshot : creates
CPU --> Memory : reads / writes
CPU --> RegisterFile : reads / writes
CPU --> ScreenDevice : output on store to FF
CPU *-- Instruction : current IR
CPU ..> ALU : arithmetic / logic
CPU ..> StepResult : returns
Memory ..> ProgramLoader : receives parsed bytes
note right of CPU
  Fetch one 16-bit instruction from
  two adjacent memory cells; PC normally
  advances by 2. Jump instructions may
  replace PC. C000 halts execution.
end note
note bottom of VoleMachine
  Loading requires an even start address
  below FF and enough memory capacity.
  Validate all instructions before execution.
end note
note bottom of ScreenDevice
  Address FF is treated as memory-mapped
  screen output, following the opcode table.
  Confirm the assignment's example using
  address 00 with the instructor/TA.
end note
@enduml
```

Before submission checklist

- Attach actual original conversation screenshots/export if required.
- Add full prompts/responses from the actual AI sessions for AI Designs 1, 2, and 3.
- Verify that the UML image matches the PlantUML source.
- Confirm FF versus 00 screen-output behavior with the official specification or TA.
- Add team cover page, work breakdown, two sample programs, and video link.